Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- PPM'in temel fikri ve context tablosu (ekrandan, 00:18:30)
  - PPM (Prediction with Partial String Matching), bir sembole olasılık verirken yalnız sembolün metindeki sıklığına değil, önündeki N sembole (context) bakar. Order-N Markov modeli kullanılır. Context yalnız geçmişi içerir; sembolün kendisi context'e dahil değildir.
  - Algoritma iki aşamalıdır: önce metin okunarak her order için context-sayı tablosu çıkarılır, sonra bu olasılıklar bir istatistiksel kodlayıcıya, burada aritmetik kodlamaya gönderilir. Kod çözme de aritmetik kodlamanın tersiyle ve aynı tablo üzerinden yapılır.
  - Örnek tablo "xyzzxyxyzzx" için Order 4'ten Order 0'a kadar sayıları gösterir (Salomon, Table 2.72). Yeni bir sembol gelince önce Order 0 sayısı artırılır, sonra Order 1, 2, 3… context'lerine bakılır; context varsa sayısı artar, yoksa yeni satır olarak eklenir.
- PPM'in çalışma prensibi: switching (ekrandan, 00:20:00)
  - PPM, sıfır olasılıklı bir context geldiğinde onu daha kısa bir context ile değiştirir (switching).
  - Order-N ile başlar. Gelen S sembolünü order-N modeline göre önceki C context'i içinde arar. Bulamazsa Order-$(N-1)$'e geçer; bu Order-0'a kadar sürer.
  - Order-0'da da sembol hiç görülmemişse sembole sabit bir olasılık atanır: $1/$alfabenin boyu (bu düzeye Order -1 denir).
  - Bulunan P olasılıklı her S sembolü aritmetik kodlamaya gönderilir.
- PPM kod çözücü ve escape sembolü (ekrandan, 00:20:00)
  - Kod çözücünün hangi aşamada switch yapıldığını bilmesi gerekir. Bunun için escape sembolü kullanılır.
  - Kodlayıcı, switch etmeden önce escape sembolünün olasılığını aritmetik kodlamaya gönderir. Escape de her context'te bir sembol gibi davranır.
- "assanissimassa" örneği, Order-2 tablo (ekrandan, 00:30:00)
  - Order 0: a 4, s 6, n 1, i 2, m 1, esc 5; toplam 19 (a = 4/19, s = 6/19, esc = 5/19).
  - Order 1: a→s 2, a→n 1, esc 2 (toplam 5); s→s 3, s→a 2, s→i 1, esc 3 (toplam 9); n→i 1, esc 1; i→s 1, i→m 1, esc 2; m→a 1, esc 1.
  - Order 2: as→s 2, esc 1 (2/3, 1/3); ss→a 2, ss→i 1, esc 2 (2/5, 1/5, 2/5); sa→n, an→i, ni→s, is→s, si→m, im→a, ma→s her biri 1 ve esc 1 (1/2, 1/2).
  - Escape'in frekansı, o context'i izleyen farklı sembol sayısıdır (PPMC yaklaşımı).
- Son context "sa" iken dört senaryo (ekrandan, 00:35:40 ve 00:42:20)
  - "n" gelirse: sa→n vardır, olasılık 1/2, $-\log_2(1/2) = 1$ bit.
  - "s" gelirse: sa→s yoktur, escape 1/2 (1 bit); Order-1'de a→s 2/5 (1,32 bit); toplam 2,32 bit.
  - "m" gelirse: escape 1/2 (1 bit), Order-1 escape 2/5 (1,32 bit), Order-0'da m 1/19 (4,25 bit); toplam 6,57 bit.
  - "d" gelirse: 1 + 1,32 + Order-0 escape 5/19 (1,93 bit) + Order -1'de 1/28 (4,8 bit) = 9,05 bit.
  - Yeni gelen sembol hemen tablolara eklenir; böylece bir sonraki gelişinde daha az bitle kodlanır.
- PPM'in versiyonları: PPMA, PPMB, PPMC (ekrandan, 00:50:00)
  - Versiyonlar yalnız escape sembolünün kullanımında ayrılır; sıkıştırma oranları arasında çok küçük farklar vardır. Derste gösterilen yöntem PPMC'dir.

  - PPMA: toplam frekans n ise escape'e $1/(n+1)$, diğer sembollere x/n olasılığı atanır (slayttaki gibi; bkz. Detaylı Açıklamalar).
  - PPMB: C context'inin içeriğinde s iki kez göründükten sonra C'yi izleyen S sembolüne olasılık atanır. "abc" context'i üç kez, onu izleyen "x" iki kez, "y" bir kez görünmüşse x için $(2-1)/3$, y için $(1-1)/3$ atanır.
- PPM'in pratikte gerçekleştirilmesi: trie (ekrandan, 00:50:00 ve 00:57:30)
  - Ana problem, input'tan okunan her sembolün 0'dan N'ye kadar bütün context'lerinin veri yapısını kurmak ve bunlara hızlı erişmektir. Tabloda sıralı arama mümkün değildir.
  - Bunun için trie ağacı kullanılır. Kök boştur; her düğüm "sembol, sayı" taşır; derinlik 1 Order-0, derinlik 2 Order-1, derinlik 3 Order-2 context'leridir.
  - "zxzyzxxyzx" için on adımlık trie örneği (Salomon, Figure 2.74) gösterildi.
- Vine pointer (ekrandan, 01:00:00)
  - Algoritmayı basitleştirmek için her düğüme geriye doğru bir sonraki kısa context'i gösteren bir pointer eklenir. Slaytta bu pointer "wine pointer" diye yazılmıştır.
  - Uzun context'te sembol bulunamazsa pointer izlenerek daha kısa context'e, en sonunda köke inilir. Bu, tablodaki escape ile switch etmenin ağaçtaki karşılığıdır. Örnek: "assanissimassa"nın ilk altı trie'si (Salomon, Figure 2.75).
- Ders içi alıştırma: Huffman ve aritmetik kodlama tekrarı (ekrandan, 01:20:00, 01:42:50 ve 01:53:30)
  - Frekansları 3, 3, 3, 6, 6, 9, 12 olan yedi sembolün Huffman kodları: 12 → `00`, 6 → `010`, 6 → `011`, 9 → `10`, 3 → `111`, 3 → `1100`, 3 → `1101`. Kodlar prefix koddur; ortalama uzunluk $111/42 \approx 2{,}64$ bit (python ile hesaplandı, derste hesaplanmadı).
  - Olasılıkları a 0,4, b 0,3, c 0,29 olan sembollerin toplamı 0,99'dur; eksik 0,01 escape (EOF) sembolüne verilir. Aralıklar: a $[0{,}6;\ 1)$, b $[0{,}3;\ 0{,}6)$, c $[0{,}01;\ 0{,}3)$, esc $[0;\ 0{,}01)$.
- Sözlük tabanlı yöntemler (ekrandan, 01:20:00 ve 02:00:00)
  - İstatistiksel yöntemler verinin istatistiksel modelini kullanır; sıkıştırmanın kalitesi seçilen modele bağlıdır. Sözlük tabanlı yöntemlerde ne istatistiksel model ne de değişken uzunluklu kod kullanılır.
  - Semboller yerine stringler ve bunları ifade eden token'lar kullanılır. Token; stringin kendisini, offset'ini (dokümandaki yerini) ya da uyuşma uzunluğunu (length) tutabilir.
  - Statik sözlük önceden hazırlanır; algoritmaya göre ekleme ya da silme yapılabilir veya yapılmaz. Dinamik sözlük input stream okundukça oluşur; ekleme ve silme yapılır.
  - Statik sözlüğün en basit örneği İngilizce dokümanlar için İngilizce sözlüktür. Kelimeler ek aldığı için Türkçe için uygun değildir.
- Statik sözlükle kodlama örneği (ekrandan, 02:00:00 ve 02:10:00)
  - Yarım milyon kelimelik sözlük için index alanı 19 bittir ($2^{19} = 524.288$). Kelime sözlükte varsa sırası, yoksa kendisi yazılır; ikisini ayırt etmek için bir flag biti kullanılır.
  - Sözlükte var: 1 bit flag (0) + 19 bit index = 20 bit. Sözlükte yok: 1 bit flag (1) + 7 bit karakter sayısı + 5 × 8 bit = 48 bit (5 karakterlik kelime).
- String sıkıştırma (ekrandan, 02:17:40, 02:20:00 ve 02:24:00)
  - Sembollerden oluşan string'lere dayanan yöntemler, tek sembole dayanan yöntemlerden daha etkilidir.
  - İki sembollü alfabe ($P_1 = 0{,}8$, $P_2 = 0{,}2$) tek tek kodlanırsa 1 bit/sembol gerekir. İkili string'lerle 0,78, üçlü string'lerle 0,728 bit/sembol olur.

Hocanın Özellikle Vurguladığı Kısımlar

- Context tablosunu doğru ve net oluşturmak
  - Hoca, PPM'de tablonun doğru kurulmasının çok önemli olduğunu söyledi; olasılıklar ve escape değerleri bu tablodan gelir.
- Escape'in değeri
  - Bir context'teki escape frekansı, o context'i izleyen farklı sembol sayısıdır; olasılıklar context'in sayıları ile escape frekansının toplamına bölünerek bulunur. Order-0'da 5 farklı sembol olduğu için escape 5, toplam 19'dur.
- Bit sayısı $-\log_2 p$ ile bulunur
  - Olasılığın 2 tabanında logaritması, o sembolü kodlamak için gereken bit sayısını verir. Switch edilen her order'ın escape biti de toplama eklenir.

  - Hoca, “m” için 6,57 bitin bile 8 bitlik ASCII’den iyi olduğunu; uygun context’lerde sembolün 1-2 bitle kodlanabileceğini, sonucun metnin içeriğine bağlı olduğunu söyledi.
- En popüler versiyon PPMC’dir
  - A ve B’den sonra geldiği için daha başarılı olduğunu, PPMB’nin context’leri saymak zorunda olduğu için daha maliyetli olduğunu söyledi.
- Huffman ağacında tutarlılık (ekrandan, 01:42:50 ve 01:47:20)
  - Toplam değer listedeki bir frekansla eşitse soluna ya da sağına konabilir, ama hep aynı tarafa konmalıdır. Ağaç kurulurken de ilk okunan hep sola (ya da hep sağa) konmalıdır.
  - Farklı ama tutarlı bir ağaç farklı kodlar verir; prefix özelliği korunuyorsa çözüm doğrudur. Entropi, ortalama kod uzunluğu ve redundancy değişmez. Her adımda taraf değiştirilirse prefix özelliği kaybolabilir.
- Sözlükte bulunma oranı sorusu (ekrandan, 02:14:30)
  - Kelimelerin yaklaşık %29’undan fazlası sözlükte bulunmazsa sıkıştırma değil genişletme yapılır. Hoca bu soruyu daha önce vizede sorduğunu söyledi.
  - Özel bir problemde (örneğin finans metinleri) birkaç yüz kelimelik statik sözlük her zaman iyi sonuç verir.
- Karakter değil, string bazında çalış
  - String uzunluğu arttıkça sembol başına ortalama kod uzunluğu düşer ve olasılıkların varyansı artar. Hoca, alfabe 29 karakter olunca kombinasyonların çok daha fazla olacağını hatırlattı.

Kısa Tekrar Notları

- PPM = context (önceki N sembol) + aritmetik kodlama. Context sembolün kendisini içermez.
- Switching: order-$N \to N-1 \to \ldots \to 0 \to -1$ ($1/$alfabe boyu).
- Her switch’ten önce o context’in escape olasılığı gönderilir.
- PPMC escape frekansı = context’i izleyen farklı sembol sayısı.
- assanissimassa: Order 0 toplamı 19 (a 4, s 6, n 1, i 2, m 1, esc 5).
- “sa” sonrası: n → 1 bit; s → 1 + 1,32 = 2,32 bit; m → 1 + 1,32 + 4,25 = 6,57 bit; d → 1 + 1,32 + 1,93 + 4,8 = 9,05 bit.
- PPMA: esc $1/(n+1)$. PPMB: context’te iki kez görünmeden olasılık atanmaz; $(x-1)/n$.
- Trie: kök boş, düğüm = (sembol, sayı); geriye işaret eden pointer’a vine pointer denir.
- Aritmetik kodlama tablosu: a 0,4 → [0,6; 1), b 0,3 → [0,3; 0,6), c 0,29 → [0,01; 0,3), esc 0,01 → [0; 0,01).
- Sözlük tabanlı: olasılık ve değişken uzunluklu kod yok; token = string / offset / length.
- Statik sözlük önceden hazır; dinamik sözlük okurken oluşur.
- 500.000 kelime → 19 bit index; var: 20 bit, yok: 48 bit (5 karakter); $p > 0{,}29$ olmalı.
- String boyu 1, 2, 3 → varyans 0,6 / 0,78 / 0,792; ortalama kod 1 / 0,78 / 0,728 bit/sembol.

Detaylı Açıklamalar

- Context tablosunun artımlı kurulması (ekrandan, 00:18:30)
  - “xyzzxyxyzzx” için tablo (a), Order 4’ten 0’a kadar şöyledir: Order 0’da x 4, y 3, z 4; Order 1’de x→y 3, y→z 2, y→x 1, z→z 2, z→x 2.
  - Ardından bir “z” daha gelince tablo (b) olur: Order 0’da z 5; Order 1’de x→z 1 eklenir; Order 2’de xy→x’in altına xy→z 1 eklenir ve bu böyle Order 4’e kadar gider.
  - Hoca örnek olarak ekranda “xxxy” yazdı: “y” geldiğinde Order 0’daki y sayısını 3’ten 4’e çıkardı, sonra x→y, xx→y, xxx→y context’lerine baktı; olmayanları 1 sayısıyla ekledi.
  - Başka bir doküman bu tabloyla kodlanırken uzun context bulunamazsa order bir azaltılarak daha kısa context aranır (örneğin “pqyz” ardından “z” gelirse önce dörtlü, sonra üçlü, sonra ikili context aranır).
- assanissimassa tablosunun kurulması (ekrandan, 00:30:00)

```
Order 2            Order 1            Order 0
as->s  2  2/3      a->s  2  2/5       a    4  4/19
esc    1  1/3      a->n  1  1/5       s    6  6/19
```

```
ss->a  2  2/5      esc   2  2/5        n    1  1/19
ss->i  1  1/5      s->s  3  3/9        i    2  2/19
esc    2  2/5      s->a  2  2/9        m    1  1/19
sa->n  1  1/2      s->i  1  1/9        esc  5  5/19
esc    1  1/2      esc   3  3/9
an->i  1  1/2      n->i  1  1/2
esc    1  1/2      esc   1  1/2
ni->s  1  1/2      i->s  1  1/4
esc    1  1/2      i->m  1  1/4
is->s  1  1/2      esc   2  2/4
esc    1  1/2      m->a  1  1/2
si->m  1  1/2      esc   1  1/2
esc    1  1/2
im->a  1  1/2
esc    1  1/2
ma->s  1  1/2
esc    1  1/2
```

- Tablonun sayıları python ile metinden yeniden çıkarıldı ve slaytla aynı bulundu.
- Hoca tabloyu ekranda sayarken bir yerde bir s'yi atladığını fark etti; slayttaki sayılar (a 4, s 6) doğrudur.
- Senaryolar ve bit hesabı
  - Bit sayıları: $-\log_2(1/2) = 1$, $-\log_2(2/5) \approx 1{,}32$, $-\log_2(1/19) \approx 4{,}25$, $-\log_2(5/19) \approx 1{,}93$, $-\log_2(1/28) \approx 4{,}81$ (python ile doğrulandı).
  - **Ders sonrası not:** Hoca "d" örneğinde Order-0 escape için "5 bölü 9" dedi; slayttaki doğru değer 5/19'dur. "1 bölü 32" dediği yer de 1,32 bittir.
  - **Ders sonrası not:** Order -1'de alfabe boyu slaytta 28 alınmıştır; hoca Türkçe için 29, başka bir yerde 26 örneğini de verdi. Değer kullanılan alfabeye göre değişir.
  - "d" kodlandıktan sonra Order-0'a eklenir: d 1, escape 6, toplam 20 olur ve d'nin olasılığı 1/20 olur. Gerekirse Order 1 ve Order 2 context'leri de güncellenir.
- PPMA olasılıkları
  - **Ders sonrası not:** Slaytta PPMA için escape $1/(n+1)$, diğer semboller x/n yazıyor. Olasılıkların toplamının 1 olması için diğer sembollere büyük olasılıkla $x/(n+1)$ atanmalıdır.
- Trie oluşturma adımları ("zxzyzxxyzx", ekrandan, 00:57:30)
  - z gelir: kökün altına (z,1). x gelir: (x,1) ve z'nin altına (x,1) yani zx. z gelir: z 2 olur, x'in altına z (xz), zx'in altına z (zxz) eklenir. y gelir: (y,1), z'nin altına y (zy), xz'nin altına y (xzy).
  - Bu şekilde onuncu adımda kök altında x 4, y 2, z 4 sayıları olur. Ağaç üzerindeki arama tablodaki sıralı aramadan çok daha hızlıdır.
- Vine pointer ile switch etme
  - "assanissimassa" örneğinde hoca, bir sembol en uzun context'te bulunamazsa pointer'ın bir kısa context'e (örneğin "as" → "s"), oradan tek sembole, en sonunda köke gittiğini gösterdi. Köke gelmek, Order-0'dan da çıkıp yeni sembol durumuna geçmek demektir.
- Statik sözlükte gereken bulunma oranı (ekrandan, 02:14:30)
  - N kelime, 5 karakterlik kelimeler, sıkıştırmasız $40N$ bit. Sözlükte bulunma olasılığı P ise:

$$N[20P + 48(1-P)] = 40N \;\Rightarrow\; 48 - 28P = 40 \;\Rightarrow\; P = \frac{8}{28} \approx 0{,}29$$

- $P > 0{,}29$ ise sıkıştırma olur; altında genişletme, tam sınırda aynı uzunluk elde edilir (python ile doğrulandı: $8/28 = 0{,}2857$).
- **Ders sonrası not:** Hoca bir yerde "sözlükte varsa 9 bitlik index" dedi; doğrusu 19 bittir. Soruyu tekrar ederken "500 bin kelime için 20 bit" demesi, 1 bitlik flag'in eklendiği toplamdır; index 19 bittir.
- String sıkıştırma tabloları (ekrandan, 02:20:00 ve 02:24:00)
  - Tek sembol: ortalama olasılık 0,5; "varyans" $|0{,}8-0{,}5|+|0{,}2-0{,}5| = 0{,}6$; 1 bit/sembol.
  - İkili: a1a1 0,64 → `0`, a1a2 0,16 → `11`, a2a1 0,16 → `100`, a2a2 0,04 → `101`. Ortalama

olasılık 0,25; varyans 0,78; $1 \times 0{,}64 + 2 \times 0{,}16 + 3 \times 0{,}16 + 3 \times 0{,}04 = 1{,}56$ bit/string, $1{,}56/2 = 0{,}78$ bit/sembol.
  - Üçlü: a1a1a1 0,512 → `0`; 0,128 olanlar `100`, `101`, `110`; 0,032 olanlar `11100`, `11101`, `11110`; a2a2a2 0,008 → `11111`. Varyans 0,792; $2{,}184$ bit/string, $0{,}728$ bit/sembol.
  - Bütün değerler python ile doğrulandı.
  - **Ders sonrası not:** Hoca "iki sembolden 2 üzeri 4" dedi; ikili string'lerin sayısı $2^2 = 4$'tür. Üçlü tabloda kod uzunlukları 1, 3 ve 5 bittir (hoca "2, 4, 5" dedi). Hocanın "0,14" ve "2,56" dediği yerler slaytta 0,04 ve 1,56'dır.
- Hecelerle ve morfolojik birimlerle kodlama
  - Hoca, Türkçe metni hecelere ayırıp heceleri sembol olarak kodlamayı örnek verdi: Türkçede hecelerin çoğu 2 ya da 3 karakterlidir, 4 karakterli hece çok azdır.
  - Kelimeleri kök ve eklerine ayırıp kökleri ve ekleri ayrı Huffman ağaçlarında kodlamanın performansı artırdığını, ama "kitabı = kitap + ı" gibi ünsüz değişmelerinin açarken geri çevrilmesi gerektiğini anlattı.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.